

REC'N'PLAY 2026 · WORKSHOP PRÁTICO · 13H-17H

Construção de Agentes Autônomos com Gemini API e Function Calling

Ferramentas externas, APIs, structured outputs e fluxos automatizados sem alucinações. Do zero, para quem está começando.

Petros Barreto · GDG Recife

QUEM VAI TE GUIAR HOJE

Petros Barreto

Mais de 10 anos desenvolvendo software, de apps iOS e multiplataforma a plataformas de cloud em escala.



GDG Recife

Organizer da comunidade Google Developer Group Recife

Magalu Cloud

Coordenador de Sistemas na Magalu Cloud

Professor

Professor Universitário na UNIT e na UNIFG

Pesquisa

Mestrando no PPGEC/UPE

NOSSA TARDE

4 horas, 5 módulos, 1 agente no seu GitHub

13:00 Boas-vindas, primeiros conceitos e setup

13:25 Fundamentos: LLM, alucinação, agente

13:40 Módulo 1 · Primeira chamada

14:10 Módulo 2 · Structured outputs

14:40 Intervalo ☕

14:55 Módulo 3 · Function calling

15:25 Módulo 4 · O agente autônomo

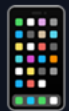
16:05 Módulo 5 · Web, Docker e GitHub

16:40 Desafios

16:50 Encerramento e limpeza do computador

TUDO DA OFICINA EM UM SÓ LUGAR

Aponte a câmera



Slides em PDF

para baixar e rever em casa



Código com botão Copiar

de todos os módulos



Template do GitHub

para criar o seu repositório



Avaliação da oficina

no final da tarde

petros.dev.br/oficina-agentes-gemini



Complete os 5 checkpoints e ganhe

- | | | |
|---|----------|----------------------------------|
| 1 | Módulo 1 | commit da primeira chamada |
| 2 | Módulo 2 | commit dos structured outputs |
| 3 | Módulo 3 | commit do function calling |
| 4 | Módulo 4 | agente respondendo + commit |
| 5 | Módulo 5 | interface web rodando + git push |

Tudo no **seu repositório no GitHub**. Mostre ao instrutor até as 16:50.



O PRÊMIO

**Ingresso 100% free
para o DevFest
Recife 2026**

+ brindes do GDG Recife 🎁

gdg.rec.br/devfest2026

CASO DE USO REAL

iD Barber

Gestão de barbearias: agendamentos, clientes e equipe. No site e nos aplicativos.

O cliente pede em linguagem natural. O agente consulta, confere e agenda de verdade, sem inventar horário.

idbarber.com.br

Quero cortar cabelo e barba sábado de manhã, pode ser com o João?

⚙️ `listar_servicos()`

→ corte + barba · 50 min

⚙️ `horarios_livres("João", "sábado")`

→ 09:00 · 10:30 · 11:10

⚙️ `criar_agendamento("João", "sáb 09:00")`

→ confirmado ✓

Pronto! Sábado às 9h com o João, corte + barba. Te lembro na véspera 

Uma foto vira a ficha técnica do corte



O QUE VOCÊ LEVA PARA CASA

O Guia Rec'n'Play

Você: Monte meu sábado com algo de IA e algo de carreira, sem conflito.
Vou precisar de guarda-chuva?

```
🔧 passo 1: consultar_programacao(dia="sábado")  
🔧 passo 1: previsao_do_tempo(cidade="Recife")  
🔧 passo 2: adicionar_na_minha_agenda("A12")  
🔧 passo 2: adicionar_na_minha_agenda("A13")
```

Guia: Pronto! Salvei A12 (10h) e A13 (14h).
Chance de chuva de [__]%; leve sim 🌂

Conversa

Entende português do jeito que a gente fala

Age

Consulta APIs reais e salva sua agenda

Não inventa

Se não tem o dado, ele diz

ANTES DE TUDO

Onde o código vive e como ele roda

1

Editor · VS Code

Onde você lê e escreve o código.
Um arquivo .py é uma receita de passos.

VOCÊ ESCREVE



2

Terminal

Onde você manda o computador seguir a receita: `python ola_gemini.py`

VOCÊ EXECUTA



3

Resultado

O que o programa mostra aparece ali mesmo, no terminal.

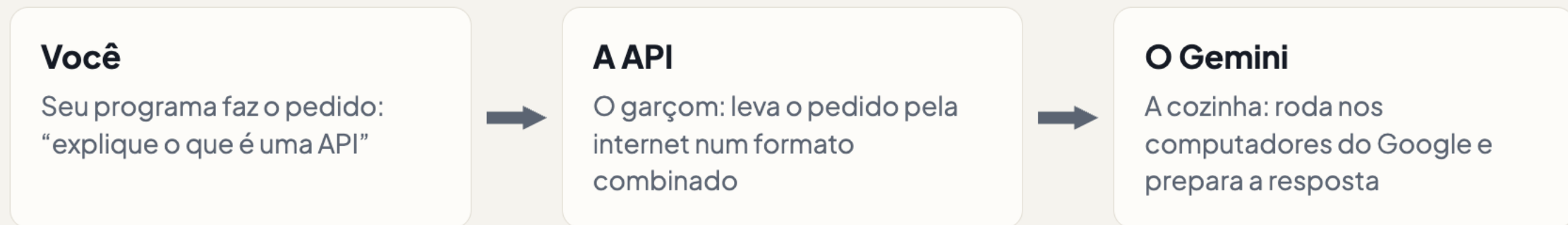
VOCÊ LÊ



No VS Code: **Arquivo** > **Abrir Pasta** para ver o projeto e **Terminal** > **Novo Terminal** para rodar os comandos.

O QUE É UMA API?

Um garçom entre o seu programa e o Gemini



A chave de API é a sua comanda: diz ao Google quem está pedindo e controla quanto você pode usar. Para começar, é gratuita.

PASSO A PASSO

Como conseguir sua chave (grátis, 2 minutos)

- 1 Abra **aistudio.google.com/apikey** no navegador (no laboratório, em janela anônima)
- 2 Entre com a sua conta Google
- 3 Aceite os termos, se aparecerem, e clique em **Create API key** (Criar chave de API)
- 4 Copie a chave: é um texto longo, parecido com **AlzaSy...**

⚠ A chave é como uma senha: não mande no grupo, não coloque em prints e não publique no GitHub.

ONDE GUARDAR A CHAVE

O arquivo .env: um cofre fora do código

```
# arquivo .env, na pasta do projeto  
GEMINI_API_KEY=AizaSy...cole_a_sua_aqui  
GEMINI_MODEL=gemini-3.8-flash
```

1 · Crie

Copie o modelo: `copy .env.example .env`

2 · Cole

Depois do `=`, sem espaços e sem aspas

3 · Proteja

O `.gitignore` impede que ele vá para o GitHub

O programa lê a chave daqui sozinho (arquivo **comum.py**). Você nunca precisa escrever a chave dentro do código.

Seu repositório desde o primeiro minuto

1

Use o template

Abra o link do QR Code
> Use this template

2

Clone o seu

git clone + o link do
SEU repositório

3

Instale

```
python -m venv .venv e  
pip install -r  
requirements.txt
```

4

Chave e teste

```
cole no .env e rode  
verificar_ambiente.py
```



github.com/petrosbarreto/oficina-agentes-gemini

Computador compartilhado: use **janela anônima** no navegador. Plano B se algo travar: **oficina_colab.ipynb** no Colab.



Fundamentos

Antes do código: o que é um modelo de linguagem, por que ele alucina e o que transforma um chatbot em agente.

Um LLM é um autocompletar gigante

"O céu de Recife hoje está _____"

ensolarado

62% de probabilidade

nublado

25% de probabilidade

roxo

0,01% de probabilidade

Ele aprendeu padrões lendo bilhões de textos. Ele **não consulta** o céu de verdade.

Três palavras que vamos usar o tempo todo



Token

Pedaço de texto que o modelo lê. Cerca de $\frac{3}{4}$ de uma palavra. Limites e custos são contados em tokens.



Prompt

O que você pede. Quanto mais claro e específico, melhor a resposta.



Modelo

O cérebro escolhido. Hoje: **gemini-3.8-flash**, rápido e barato.

O PROBLEMA

Alucinação: confiança sem conhecimento

PERGUNTA

"Qual oficina acontece às 14h na sala Maracatu?"



RESPOSTA POSSÍVEL

"A oficina de Blockchain com Maria Souza!" 🤖

Ele não tem acesso à programação, ao relógio, nem à internet. Mas sempre responde algo **plausível**.

A DIFERENÇA

Chatbot responde. Agente resolve.

	Chatbot	Agente
Conhece	Só o que aprendeu no treino	O que as ferramentas trazem agora
Faz	Escreve texto	Consulta, calcula, salva, envia
Decide	Uma resposta por vez	Vários passos até cumprir o objetivo
Erra por	Inventar	Ferramenta ruim ou regra frouxa

Agente = LLM + ferramentas + laço + objetivo



LLM

O cérebro que entende e decide



Ferramentas

As mãos: funções e APIs



Laço

Repete até terminar



Objetivo

O pedido do usuário

Analogia: um estagiário esperto (LLM) com telefone e lista de contatos (ferramentas), que liga quantas vezes precisar (laço) até resolver seu pedido (objetivo).

01

Primeira chamada

Falar com o Gemini pelo Python, entender a resposta e dar memória à conversa.

Só o que você precisa para ler o código de hoje

`# um lembrete`

Comentário. Começa com `#`.
O computador ignora; serve
para explicar.

`"Olá, Recife"`

Texto. Tudo entre aspas é usado
exatamente como está escrito.

`nome = "Ana"`

Variável. Uma caixinha com nome
que guarda um valor. `=` quer
dizer guarde.

`print("oi")`

Função. Um comando pronto. Nos
parênteses vai o que ele precisa.
`print` mostra na tela.

`from google import genai`

Import. Pega uma caixa de
ferramentas que alguém já fez.

`cliente.interactions.create()`

Ponto. Entra em algo: do cliente,
pegue interactions e use create.

Cinco linhas para falar com o Gemini

```
from google import genai

cliente = genai.Client(api_key="SUA_CHAVE")

interaction = cliente.interactions.create(
    model="gemini-3.8-flash",
    input="Explique o que é uma API em 3 frases.",
)

print(interaction.output_text)
```

Client

Conecta com sua chave

model

Qual cérebro usar

input

Seu pedido (prompt)

"SUA_CHAVE" está aí só para ficar didático. No projeto, a chave vem do arquivo .env.

O que cada parte do código faz

<code>from google import genai</code>	Pega a biblioteca do Google que sabe conversar com o Gemini
<code>cliente = genai.Client(api_key=...)</code>	Cria o cliente: a conexão com o Google, usando a sua chave
<code>interaction = cliente.interactions.create(</code>	Envia o pedido ao Gemini e guarda a resposta na variável <code>interaction</code>
<code> model="gemini-3.8-flash",</code>	Escolhe qual modelo vai responder
<code> input="Explique o que é uma API..."</code>	O pedido em si, escrito em português: o prompt
<code>print(interaction.output_text)</code>	Mostra na tela o texto que o Gemini escreveu

Do comando ao resultado na tela

```
(.venv) > python 01_primeira_chamada/ola_gemini.py
```

Uma API é um jeito combinado de um programa pedir algo a outro.

É como um garçom: você faz o pedido, ele leva e traz a resposta...

Passos devolvidos pela API:

- model_output

ID desta interação: [um código gerado pela API]

(.venv)

No começo da linha = ambiente ativado. Sem ele, dá erro de biblioteca.

Muda sempre

O texto é gerado na hora: cada execução responde diferente.

Deu erro?

Leia a última linha da mensagem e veja docs/TROUBLESHOOTING.md.

Uma interaction é mais do que texto



id

Identifica esta conversa.
Usamos para continuar de onde parou.



steps

Os passos: pensamento,
pedido de função, texto final.



output_text

Atalho para o texto final que o
modelo escreveu.

Os **steps** são o segredo do agente: é por eles que vamos saber quando o modelo quer usar uma ferramenta.

O modelo não lembra. Você liga as conversas.

```
interaction = cliente.interactions.create(  
    model=MODELO,  
    input=pergunta,  
    system_instruction="Você é um monitor simpático do Rec'n'Play...",  
    previous_interaction_id=id_anterior, # continua a conversa  
)  
id_anterior = interaction.id # guarda para a próxima
```

system_instruction define o comportamento. Envie em toda chamada: ela não é herdada.

Ao vivo: pedindo o que ele não sabe



Sem regra

Pode inventar uma oficina, um horário, uma temperatura.



Com regra de honestidade

Diz 'não tenho essa informação'. Melhor, mas continua sem o dado.



Com ferramentas

Busca o dado real e responde certo. É para lá que vamos.

02

Structured outputs

Transformar texto bagunçado em dados que o seu programa consegue usar, com garantia de formato.

O PROBLEMA

Texto é ótimo para gente, péssimo para programa

"Galeraaaa!! 🚀 Amanhã tem oficina massa: Construindo agentes com Gemini, com a Ana Beatriz. Começa 1 da tarde e vai até às 5, na sala Frevo. É pra quem tá começando!"



```
{  
  "titulo": "Construindo agentes...",  
  "tipo": "oficina",  
  "horario_inicio": "13:00",  
  "duracao_minutos": 240,  
  "local": "Sala Frevo",  
  "publico_iniciante": true  
}
```

1. Descreva o formato com Pydantic

```
from pydantic import BaseModel, Field
from typing import Literal

class Palestra(BaseModel):
    titulo: str = Field(description="Título da atividade")
    tipo: Literal["palestra", "oficina", "painel", "outro"]
    horario_inicio: str = Field(description="Formato HH:MM (24h)")
    duracao_minutos: int
    local: str
    publico_iniciante: bool
```

Literal limita as opções. **description** explica o campo ao modelo e melhora muito o resultado.

2. Exija o formato e valide

```
interaction = cliente.interactions.create(  
    model=MODELO,  
    input=f"Extraia a atividade deste texto: {TEXT0}",  
    response_format={  
        "type": "text",  
        "mime_type": "application/json",  
        "schema": Palestra.model_json_schema(),  
    },  
)  
palestra = Palestra.model_validate_json(interaction.output_text)  
print(palestra.local) # "Sala Frevo"
```

A API garante JSON no formato. O Pydantic confere os tipos. Você recebe um objeto Python pronto.

O modelo interpreta. O código decide.



Regras de negócio ficam no **seu** código: previsíveis, testáveis e auditáveis. Essa ideia é a base do agente.



Intervalo

15 minutos · voltamos às **14:55**

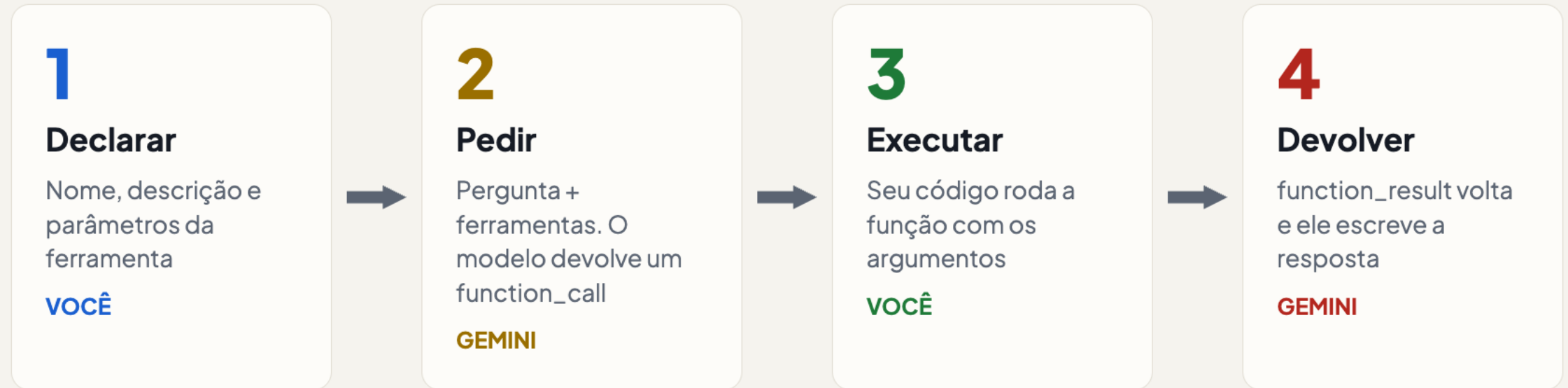
Travou em algo? Chame um monitor agora.

03

Function calling

Dar ferramentas ao modelo: ele pede, o seu código executa, e a resposta vem com dados reais.

Os 4 passos do function calling



O modelo nunca executa nada. Ele só pede.

Passo 1: o cardápio da ferramenta

```
DECLARACAO_CLIMA = {  
    "type": "function",  
    "name": "previsao_do_tempo",  
    "description": "Retorna o clima ATUAL de uma cidade brasileira. "  
                  "Use quando perguntarem sobre tempo, calor ou chuva.",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "cidade": {"type": "string", "description": "Ex.: Recife"}  
        },  
        "required": ["cidade"],  
    },  
}
```

Passos 2, 3 e 4 no código

```
# 2. Pergunta + ferramentas
interaction = cliente.interactions.create(
    model=MODELO, input=pergunta, tools=[DECLARACAO_CLIMA])

for pedido in interaction.steps:
    if pedido.type == "function_call":
        resultado = previsao_do_tempo(**pedido.arguments) # 3. nós executamos
        resultados.append({"type": "function_result", "call_id": pedido.id,
                           "name": pedido.name, "result": [...]}))

# 4. Devolver e receber a resposta final
final = cliente.interactions.create(model=MODELO, input=resultados,
    tools=[DECLARACAO_CLIMA], previous_interaction_id=interaction.id)
```

Detalhes que fazem a diferença

Descrições claras

Descrição vaga gera decisão vaga. Diga quando usar e quando não usar.

tools em toda chamada

Ferramentas e `system_instruction` não são herdadas pelo `previous_interaction_id`.

Vários pedidos de uma vez

O modelo pode pedir funções em paralelo. Execute todas antes de devolver.

Tipos e enum

Use `enum` e `required` para o modelo mandar argumentos válidos.

04

O agente autônomo

Juntar tudo: laço de decisão, cinco ferramentas, regras anti-alucinação e resposta estruturada.

As 5 ferramentas do Guia Rec'n'Play



**consultar_
programacao**

Programação do
evento

JSON local



**previsao_
do_tempo**

Clima e previsão

Open-Meteo



**buscar_
endereco_por_cep**

CEP para endereço

ViaCEP



**adicionar_
na_minha_agenda**

Salva, recusa
conflitos

Ação



**ver_
minha_agenda**

Lista o que foi salvo

Arquivo local

Cada ferramenta = **função Python + declaração**. Todas validam entradas e devolvem **{"erro": ...}** em vez de quebrar.

O laço: o que torna isto um agente

```
def perguntar(self, pergunta):
    interaction = self._chamar_modelo(pergunta)

    for passo in range(1, MAX_PASSOS + 1): # limite de segurança
        pedidos = [s for s in interaction.steps if s.type == "function_call"]
        if not pedidos: # nada a pedir: terminou!
            return interaction.output_text

        resultados = [executar(p) for p in pedidos] # nosso código executa
        interaction = self._chamar_modelo(resultados) # modelo decide o próximo

    return "Não consegui concluir dentro do limite de passos."
```

SEM ALUCINAÇÕES

Seis defesas do nosso agente



Fonte única

Fatos só vêm das ferramentas
(system_instruction)



Validação

CEP com 8 dígitos, id que existe



Regra de negócio

Conflito de horário é recusado
pelo código



Erro vira dado

`{"erro": ...}` volta para o
modelo admitir



Limite de passos

MAX_PASSOS = 8 evita laço infinito



Auditoria

Cada passo aparece no terminal

Uma ferramenta que não se deixa enganar

```
def adicionar_na_minha_agenda(id_atividade: str) → dict:
    atividades = {a["id"]: a for a in _carregar_atividades()}

    if id_atividade not in atividades: # não aceita id inventado
        return {"erro": "Não existe essa atividade. Consulte a programação."}

    conflitos = [...] # checa sobreposição
    if conflitos:
        return {"ok": False, "mensagem": "Conflito de horário.", "conflita_com": conflitos}

    _salvar_agenda(agenda + [id_atividade])
    return {"ok": True, "adicionada": atividades[id_atividade]}
```

Roteiro de teste ao vivo

- 1 Quais oficinas de IA tem na sexta?
- 2 Vai chover em Recife nos próximos dias?
- 3 Oficinas para iniciantes na quinta; adiciona a de agentes
- 4 Adiciona também a A03 → conflito!
- 5 Adiciona a Z99 → id inexistente
- 6 Monte meu sábado com IA e carreira, sem conflito, e diga se levo guarda-chuva
- 7 /json → a mesma resposta estruturada

Testando o agente sem gastar nenhuma chamada

```
class ClienteFalso:
    """Finge ser a API: 1º pede ferramenta, 2º responde."""

def test_laco_do_agente():
    agente = Agente(ClienteFalso())
    resposta = agente.perguntar("O que tem no sábado?")
    assert "A13" in resposta
    assert agente.ferramentas_usadas == ["consultar_programacao"]
```



pytest -q

7 testes, sem chave e sem internet



Por quê?

O modelo é imprevisível. As ferramentas não podem ser.

05

Do terminal para o mundo

Interface web, container Docker e código no seu GitHub. Você sai daqui com um projeto seu, rodando.

A mesma inteligência, agora com cara de app



A chave fica no servidor. Tudo que chega ao navegador qualquer pessoa vê apertando F12.

Uma rota, e o agente vira serviço

```
@app.post("/api/chat")
def conversar(p: Pergunta):
    s = _obter_sessao(p.sessao) # cada aba, um agente
    resposta = s["agente"].perguntar(p.mensagem)
    return {"resposta": resposta,
            "passos": s["agente"].passos}
```

Rodar

uvicorn app:app --app-dir 05_web -
-reload

Abrir

localhost:8000

Ver

Cada resposta mostra as
ferramentas usadas

Empacotando o agente num container

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER agente
CMD uvicorn app:app --app-dir 05_web ...
```

```
docker build -t guia-recnplay .

docker run --rm -p 8000:8000 \
  --env-file .env guia-recnplay

# ou: docker compose up --build
```

Roda igual no laboratório, na sua casa e na nuvem. O **.dockerignore** impede que o `.env` entre na imagem.

SEU CÓDIGO, SUA CONTA

Commit a cada módulo, push antes de sair

```
# uma vez, só neste repositório (o computador é compartilhado!)
git config --local user.name "Seu Nome"
git config --local user.email "seu_usuario@users.noreply.github.com"

# ao fim de cada módulo
git add .
git commit -m "módulo 4: meu agente com 5 ferramentas"
git push
```

O **.gitignore** protege o **.env**: sua chave nunca vai para o GitHub. Confira com **git status** antes do commit.

Um link público para mostrar a todo mundo

1

Crie um Space

huggingface.co › New Space ›
Docker (grátis)

NAVEGADOR



2

Guarde a chave

Settings › Secrets ›
GEMINI_API_KEY

NAVEGADOR



3

Envie o código

git remote add space ... e git push
space main

TERMINAL

Link público usa a **sua** cota: deixe o Space privado ou mantenha o limite de mensagens. Passo a passo em **docs/DEPLOY.md**.

Sua vez: escolha um desafio

★ Remover da agenda

Nova ferramenta com validação

★ Filtro por nível

Novo parâmetro com enum

★★ Cotação de moeda

API externa AwesomeAPI

★★★ Confirmação humana

Human-in-the-loop antes de agir

★★★★ Exportar .ics

Agenda no Google Agenda

★★★★★ Google Search nativo

Misturar ferramentas nativas e suas

Quem completou a missão?



Repositório seu no GitHub

criado a partir do template, na sua conta



Um commit por módulo

5 commits visíveis no histórico (git log)



Agente respondendo na web

localhost:8000 ou docker compose up



Nenhuma chave no repositório

o .env fica só no seu computador



Ingresso do DevFest Recife 2026 + brindes

Levante a mão e mostre seu
repositório

PARA LEVAR PARA A VIDA

O modelo **decide.**

O código **executa.**

Os dados **provam.**

Leve o código, deixe o computador limpo

- 1 **git status** sem pendências e **git push** feito
- 2 **python 00_setup/limpar_computador.py** (apaga o .env e desconecta o GitHub)
- 3 Sair da conta Google e do GitHub no navegador
- 4 Abrir seu repositório no celular e conferir que está tudo lá 

Computadores de laboratório podem ser restaurados ao reiniciar: o que não estiver no GitHub se perde.

OBRIGADO! 💙

Continue construindo com a gente



Avalie a oficina

wkf.ms/4dgVQzo

Material da oficina

petros.dev.br/oficina-agentes-gemini · slides em PDF e código para copiar

GDG Recife

Encontros, DevFest e comunidade

Contato

linkedin.com/in/petrosbarreto

petros.dev.br